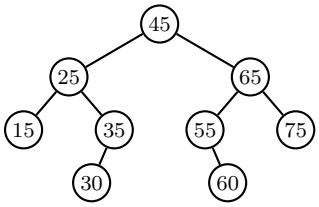


Tree T



Tree P (insert 45, 25, 65, 15, 35, 55, 75, 30, 60)

value moved / link changed

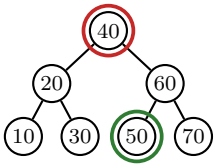
node being deleted

Programs on tree P build it with the usual BST `insert` (shown in Case 1, Q4). Programs for questions that ask for a drawing also print the preorder, which together with the inorder fixes the shape of the tree.

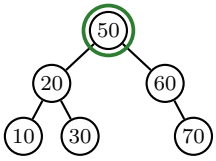
Case 1: Deleting the Root (case1.cpp)

Rules: B5, B6

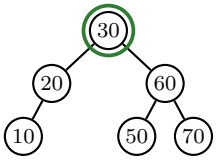
Q1 Visited: 40 (`parent = root`) → 60 (`succ = root->right`) → 50 (60 has a left child, so `parent = 60`, `succ = 50`). 50 has no left child, so the loop stops. Successor = 50, its parent = 60.



Q2 Inorder: 10 20 30 50 60 70. Preorder: 50 20 10 30 60 70.
The inorder is strictly increasing, which is exactly B7: the new root 50 is larger than everything on its left (10, 20, 30) and smaller than everything on its right (60, 70).



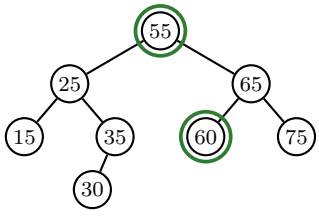
Q3 Predecessor: left once to 20, then right as far as possible to 30 (`parent = 20`). 30 is copied into the root and `20->right = 30->left = NULL`. **New root** = 30**.** The inorder does *not* change (10 20 30 50 60 70): it always lists the remaining keys in sorted order, and both versions keep the same keys in a valid BST. Only the shape changes (preorder 30 20 10 60 50 70).



Changed in `case1_q3.cpp`:

```
Node* pred = root->left;                // go left once (20)
while (pred->right != nullptr)          // then right as far as possible
{
    parent = pred;
    pred = pred->right;                 // pred = 30, parent = 20
}
root->data = pred->data;                // root now holds 30
if (parent == root)
    parent->left = pred->left;
else
    parent->right = pred->left;         // 20->right = NULL
delete pred;
```

Q4 Successor of 45: right once to 65, then left to 55 (no left child). Successor = 55, parent = 65. 55 is copied into the root. 55 is *not* a leaf: it has a right child, 60. So it is removed with **B4**, not B3: `65->left = 55->right = 60`. The same line `parent->left = succ->right` covers both rules (`succ->right` is NULL for B3).
Inorder: 15 25 30 35 55 60 65 75. Preorder: 55 25 15 35 30 65 60 75.



Changed in `case1_q4.cpp`*:*

```cpp
Node* insert(Node* node, int key)        // usual BST insert
{
    if (node == nullptr) return new Node(key);
    if (key < node->data) node->left = insert(node->left, key);
    else                  node->right = insert(node->right, key);
    return node;
}
    int arr[] = {45, 25, 65, 15, 35, 55, 75, 30, 60};   // tree P (in main)
    int n = sizeof(arr) / sizeof(arr[0]);
    Node* root = nullptr;
    for (int i = 0; i < n; i++)
        root = insert(root, arr[i]);
```

`case1`

```
Before: 10 20 30 40 50 60 70 (root = 40)
After: 10 20 30 50 60 70 (root = 50)
```

`case1_q2`

```
Before: 10 20 30 40 50 60 70 (root = 40)
After: 10 20 30 50 60 70 (root = 50)
Preorder after: 50 20 10 30 60 70
```

`case1_q3`

```
Before: 10 20 30 40 50 60 70 (root = 40)
After: 10 20 30 50 60 70 (root = 30)
Preorder after: 30 20 10 60 50 70
```

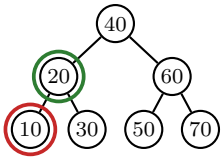
`case1_q4`

```
Before: 15 25 30 35 45 55 60 65 75 (root = 45)
After: 15 25 30 35 55 60 65 75 (root = 55)
Preorder after: 55 25 15 35 30 65 60 75
```

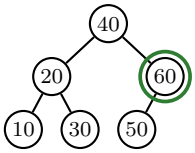
Case 2: Deleting a Leaf (case2.cpp)

Rules: B1, B3

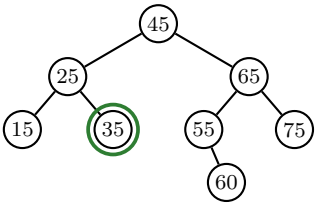
Q1 At 40: $10 \neq 40$, $10 < 40 \rightarrow$ left. At 20: $10 \neq 20$, $10 < 20 \rightarrow$ left. At 10: $10 = 10 \rightarrow$ stop.
Parent = 20, and 10 is its **left** child, so `20->left = NULL`.



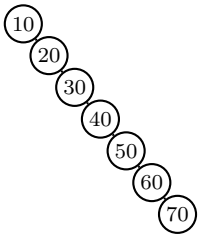
Q2 Changed `int key = 70;`. Search: 40 → right, 60 → right, 70 found with parent 60. 70 is 60's right child, so `60->right = NULL`. Printed: `After: 10 20 30 40 50 60 (root = 40)`.



Q3 Leaves of P: 15, 30, 60, 75. Deleting 30: 45 → left, 25 → right, 35 → left, found with parent 35, so `35->left = NULL`. **35 has just become a leaf.** Inorder: 15 25 35 45 55 60 65 75. (Changed: tree P, `int key = 30;`.)



Q4 The search compares the key with one node per level along a root-to-leaf path, so the worst case is $\mathbf{h+1}$ comparisons (h = height in edges). Tree T: $h = 2$, so at most 3. Inserting 10, 20, ..., 70 in order gives a chain with $h = 6$, and deleting the leaf 70 takes 7 comparisons (measured by `case2_q4.cpp`). The unlinking itself is $O(1)$, so deleting a leaf costs $O(h)$: $\Theta(\log n)$ in a balanced tree but $\Theta(n)$ when the tree degenerates into a chain.



insert 10, 20, ..., 70: $h = 6$

Changed in `case2_q4.cpp` (search loop of deleteLeaf, now counting):

```
while (cur != nullptr)
{
    comparisons++;                      // compare key with cur->data
    if (key == cur->data) break;
    parent = cur;
    if (key < cur->data) cur = cur->left;
    else                 cur = cur->right;
}
```

case2

```
Before: 10 20 30 40 50 60 70 (root = 40)
After: 20 30 40 50 60 70 (root = 40)
```

case2_q2

```
Before: 10 20 30 40 50 60 70 (root = 40)
After: 10 20 30 40 50 60 (root = 40)
```

case2_q3

```
Before: 15 25 30 35 45 55 60 65 75 (root = 45)
After: 15 25 35 45 55 60 65 75 (root = 45)
Preorder after: 45 25 15 35 65 55 60 75
```

case2_q4

```
Tree T: h = 2
Before: 10 20 30 40 50 60 70 (root = 40)
After: 10 20 30 40 50 60 (root = 40)
Deleting leaf 70: 3 comparisons (h + 1 = 3)

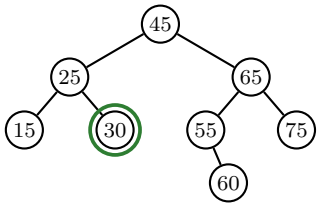
Chain: h = 6
Before: 10 20 30 40 50 60 70 (root = 10)
After: 10 20 30 40 50 60 (root = 10)
Deleting leaf 70: 7 comparisons (h + 1 = 7)
```

Case 3: Deleting a Node with One Child (case3.cpp)

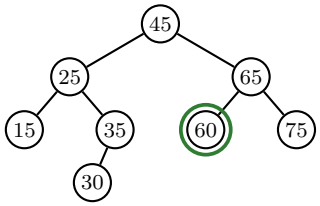
Rules: B1, B4 (B6)

Q1 **35** has one child, 30, a **left** child. **55** has one child, 60, a **right** child. Every other node of P has 0 or 2 children.

Q2 Parent = 25; child moving up = 30; pointer changed: `25->right = 30` (35 was 25's right child).
Inorder: 15 25 30 45 55 60 65 75.

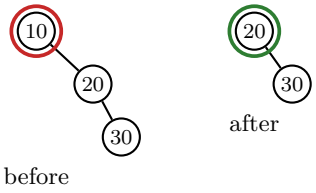


Q3 Parent = 65; child moving up = 60; pointer changed: `65->left = 60`.
Inorder: 15 25 30 35 45 60 65 75.
55 < 65, so 55 is 65's *left* child: `parent->left == cur` is true and the program writes `parent->left`. For 35 the test was false, because 35 > 25 made it 25's right child.



(Changed in `case3_q2.cpp` / `case3_q3.cpp`: tree P, `int key = 35;` / `55;`, and the "remove 50 first" step is dropped.)

Q4 B6: the root has 0 or 1 child (here one, 20), so the root pointer itself must change. **New root = 20.** In unchanged `case3.cpp`, 10 matches at the root, so the loop body never runs and `parent` stays `nullptr`. The line `if (parent->left == cur)` then dereferences NULL: undefined behaviour, and `case3_q4_unfixed.cpp` crashes with a segmentation fault. Even without the crash, `root` would still point to the freed node.



Changed in `case3_q4.cpp`:

```
if (parent == nullptr)        root = child;          // B6: the root pointer changes
else if (parent->left == cur) parent->left = child;
else                          parent->right = child;
```

case3

```
Before: 10 20 30 40 50 60 70 (root = 40)
Middle: 10 20 30 40 60 70 (60 now has one child, 70)
After: 10 20 30 40 70 (root = 40)
```

case3_q2

```
Before: 15 25 30 35 45 55 60 65 75 (root = 45)
After: 15 25 30 45 55 60 65 75 (root = 45)
Preorder after: 45 25 15 30 65 55 60 75
```

case3_q3

```
Before: 15 25 30 35 45 55 60 65 75 (root = 45)
After: 15 25 30 35 45 60 65 75 (root = 45)
Preorder after: 45 25 15 35 30 65 60 75
```

case3_q4

```
Before: 10 20 30 (root = 10)
After: 20 30 (root = 20)
Preorder after: 20 30
```

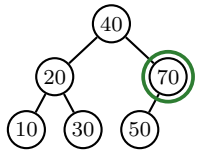
case3_q4_unfixed

```
Before: 10 20 30 (root = 10)
(crashes: segmentation fault)
```

Case 4: Deleting a Node with Two Children (case4.cpp)

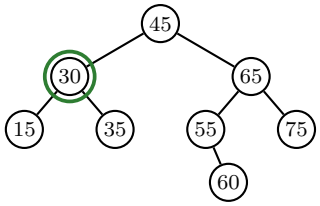
Rules: B1, B5, then B3/B4

Q1 Changed `int key = 60;`. `succ = 60->right = 70`, and 70 has no left child, so the loop never runs: **succ = 70, parent = 60 (= cur)**. The branch `if (parent == cur)` runs, because the successor is cur's own right child, not the left child of a deeper node: `cur->right = 70->right = NULL`. The node that held 60 now holds 70. Printed: `After: 10 20 30 40 50 70 (root = 40)`.

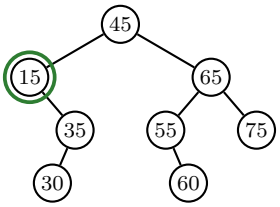


Q2 The search for the successor goes left until the left link is NULL, so it can only stop at a node with no left child. Put another way: if `succ` had a left child L, then L would also be in cur's right subtree and smaller than `succ`, so L would be the successor instead. If it could have one, step 4 would re-link only `succ->right`. `succ->left` and everything below it would be cut off: those keys would be lost from the tree and their memory leaked.

Q3 Successor of 25: right once to 35, then left to 30 (no left child). **Successor** = 30**, parent** = 35**.** 30 is copied into the node that held 25, and `35->left = 30->right = NULL` (B3).
Inorder: 15 30 35 45 55 60 65 75.



Q4 Predecessor of 25: left once to 15, which has no right child, so pred = 15 and parent = cur. 15 is copied into the node, and `cur->left = 15->left = NULL`. Inorder: 15 30 35 45 55 60 65 75.
Not the same tree as Q3: the left subtree is rooted at 15 instead of 30 (preorder 45 15 35 30 … vs. 45 30 15 35 …). **Both are valid BSTs**, because both inorders are increasing (B7).



Changed in `case4_q4.cpp`:

```
Node* pred = cur->left;                 // 15
while (pred->right != nullptr)
{
    parent = pred;
    pred = pred->right;
}
cur->data = pred->data;                 // the node that held 25 now holds 15
if (parent == cur)
    parent->left = pred->left;          // 15 was cur's left child
else
    parent->right = pred->left;
delete pred;
```

case4

```
Before: 10 20 30 40 50 60 70 (root = 40)
After: 10 30 40 50 60 70 (root = 40)
```

case4_q1

```
Before: 10 20 30 40 50 60 70 (root = 40)
After: 10 20 30 40 50 70 (root = 40)
```

case4_q3

```
Before: 15 25 30 35 45 55 60 65 75 (root = 45)
After: 15 30 35 45 55 60 65 75 (root = 45)
Preorder after: 45 30 15 35 65 55 60 75
```

case4_q4

```
Before: 15 25 30 35 45 55 60 65 75 (root = 45)
After: 15 30 35 45 55 60 65 75 (root = 45)
Preorder after: 45 15 35 30 65 55 60 75
```

Case 5: The Key Is Not in the Tree (case5.cpp)

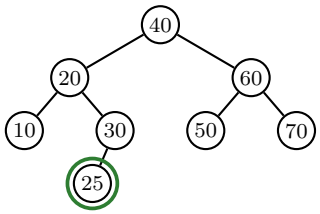
Rules: B1, B2

Q1–Q2

Key	Search path	Comp.	Stops at
25	40 L, 20 R, 30 L, NULL	3	`30->left`
5	40 L, 20 L, 10 L, NULL	3	`10->left`
45	40 R, 60 L, 50 L, NULL	3	`50->left`
75	40 R, 60 R, 70 R, NULL	3	`70->right`

All four tie at $3 = h + 1$. T is perfect with $h = 2$, so every NULL link hangs below a leaf at depth h, and an unsuccessful search always walks a full root-to-leaf path. In any BST it makes at most $h + 1$ comparisons: fast in a balanced tree, slow in a tall one.

Q3 The loop now also remembers `parent`. When `cur` reaches NULL, the new node goes on the side chosen by the last comparison: $25 < 30$, so `30->left = 25`.
Inorder: 10 20 25 30 40 50 60 70.



Changed in `case5_q3.cpp` (each branch also prints where the key went):

```
Node* parent = nullptr;                  // remember the parent this time
while (cur != nullptr && cur->data != key)
{
    parent = cur;
    ...
}
if (cur == nullptr)
{
    Node* newNode = new Node(key);
    if (parent == nullptr)       root = newNode;            // empty tree
    else if (key < parent->data) parent->left = newNode;    // 30->left = 25
    else                         parent->right = newNode;
}
```

Q4 With `root = nullptr`, `root->data` dereferences a null pointer: undefined behaviour, in practice a crash. The check below prints `the tree is empty` and stops before anything reads `root->data`. Rule **B2**: the search reaches NULL at once, so the key is not found and nothing changes.

(empty)

Changed in `case5_q4.cpp`:

```
Node* root = nullptr;                    // empty tree
if (root == nullptr)
{
    cout << "the tree is empty" << endl;
    return 0;
}
```

case5

```
Before: 10 20 30 40 50 60 70 (root = 40)
25 is not in the tree: nothing to delete
After: 10 20 30 40 50 60 70 (root = 40)
```

case5_q2

```
Before: 10 20 30 40 50 60 70 (root = 40)
25: 40 L, 20 R, 30 L, NULL -> 3 comparisons, stops at
30->left
5: 40 L, 20 L, 10 L, NULL -> 3 comparisons, stops at
10->left
45: 40 R, 60 L, 50 L, NULL -> 3 comparisons, stops at
50->left
75: 40 R, 60 R, 70 R, NULL -> 3 comparisons, stops at
70->right
After: 10 20 30 40 50 60 70 (root = 40)
```

case5_q3

```
Before: 10 20 30 40 50 60 70 (root = 40)
25 is not in the tree: inserted at 30->left
After: 10 20 25 30 40 50 60 70 (root = 40)
Preorder after: 40 20 10 30 25 60 50 70
```

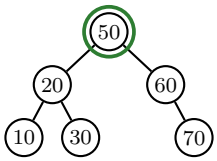
case5_q4

```
the tree is empty
```

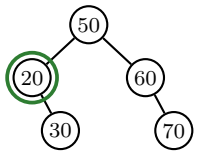
Bonus: One Function for Every Case (bonus.cpp)

Rules: B1–B6

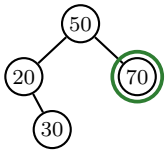
```
Node* deleteNode(Node* root, int key)
{
    Node* parent = nullptr;                 // B1: search, remembering the parent
    Node* cur = root;
    while (cur != nullptr && cur->data != key)
    {
        parent = cur;
        if (key < cur->data) cur = cur->left;
        else                 cur = cur->right;
    }
    if (cur == nullptr)                     // B2: not found (also the empty tree)
    {
        cout << key << " is not in the tree: nothing to delete" << endl;
        return root;
    }
    if (cur->left != nullptr && cur->right != nullptr)   // B5: two children
    {
        Node* succParent = cur;
        Node* succ = cur->right;
        while (succ->left != nullptr)
        {
            succParent = succ;
            succ = succ->left;
        }
        cur->data = succ->data;
        parent = succParent;                // now remove succ (it has no left child)
        cur = succ;
    }
    Node* child = (cur->left != nullptr) ? cur->left : cur->right;   // B3 / B4
    if (parent == nullptr)        root = child;                       // B6
    else if (parent->left == cur) parent->left = child;
    else                          parent->right = child;
    delete cur;
    return root;
}
```



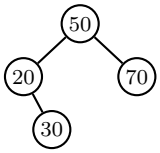
delete 40 (B5, B6)



delete 10 (B3)



delete 60 (B4)



delete 25 (B2): unchanged

Complexity. The search goes down one root-to-leaf path, and the successor search only continues down the same path, so at most $h + 1$ nodes are visited in total. Re-linking is $O(1)$. **Time** $O(h)$: $\Theta(\log n)$ for a balanced tree and $\Theta(n)$ for a chain. Extra space is $O(1)$.

bonus

```
Start: 10 20 30 40 50 60 70 (root = 40)
Delete 40: 10 20 30 50 60 70 (root = 50)
Delete 10: 20 30 50 60 70 (root = 50)
Delete 60: 20 30 50 70 (root = 50)
25 is not in the tree: nothing to delete
Delete 25: 20 30 50 70 (root = 50)

Chain: 10 20 30 (root = 10)
Delete 10: 20 30 (root = 20)
Delete 20: 30 (root = 30)
Delete 30: (empty)
30 is not in the tree: nothing to delete
Delete 30: (empty)
```